

GOLD-001 — Batch 004

Source-Integrity Review & Owner QC Packet

Production RAG v1 · evidence-candidate authoring · prepared for independent verification · 2026-08-21T16:40:08Z

WHAT IS BEING ASKED OF A REVIEWER

Fifteen candidates await a project-owner decision. Nothing here is verified, approved or eligible: every candidate is `candidate_unverified`, all 15 decisions are `null`, and no script in this repository can produce `human_verified`. The confirmed holdout-eligible count is **53**, unchanged, from batches 001–003.

The internal review below was done by the authoring model against the frozen corpus. It is a self-check, not independent verification, and that is the gap this document exists to close. Where it repaired a candidate, both the original text and the original anchor are reproduced so a disagreement is checkable without the repository.

15

candidates awaiting an owner decision

10 / 1

repaired / recommended for rejection

15 / 15

precheck holdout-ready before and after review

0

human_verified nothing is gold

1. The finding that matters most

All fifteen candidates were `precheck_holdout_ready` before this review, and all fifteen still are. The review nonetheless found one candidate to recommend for rejection and ten to repair.

What the structural precheck passed: a rule that holds only on one experimental API surface; three questions broader than the evidence answering them; four anchors whose scope lived in a section heading rather than in the span; and two critical strings that were 60-character cuts through a markdown link. The precheck verifies hashes, offsets and string containment. It cannot read.

THE THREE STATES, KEPT APART

`precheck_holdout_ready` — a script decides. The record is structurally capable.

`human_verified` — only the project owner decides. A person read the evidence and approved it.

`holdout_eligible` — derived: `human_verified` **and** deterministic claim support **and** valid evidence **and** no unresolved blocker.

Collapsing the first into the second is the failure this pipeline is built to prevent, which is why a 15/15 precheck result appears here next to a review that rejected one and repaired ten.

2. Review outcome

candidate	provider	reasoning type	shape	internal status	repaired	findings
GOLD-B004-01	anthropic	error_behavior	single_span	NEEDS_REPAIR	yes	2
GOLD-B004-02	anthropic	configuration_interaction	single_span	READY_FOR_OWNER_REVIEW	no	1
GOLD-B004-03	anthropic	error_behavior	single_span	READY_FOR_OWNER_REVIEW	no	0
GOLD-B004-04	anthropic	error_behavior	single_span	NEEDS_REPAIR	yes	2

candidate	provider	reasoning type	shape	internal status	repaired	findings
GOLD-B004-05	anthropic	error_behavior	single_span	READY_FOR_OWNER_REVIEW	no	1
GOLD-B004-06	anthropic	exact_lookup	single_span	NEEDS_REPAIR	yes	3
GOLD-B004-07	anthropic	lifecycle_compatibility_migration	single_span	NEEDS_REPAIR	yes	1
GOLD-B004-08	openai	ambiguity_disambiguation	multi_span	REJECT_RECOMMENDED	no	3
GOLD-B004-09	openai	ambiguity_disambiguation	multi_span	NEEDS_REPAIR	yes	2
GOLD-B004-10	openai	configuration_interaction	single_span	NEEDS_REPAIR	yes	2
GOLD-B004-11	openai	error_behavior	single_span	NEEDS_REPAIR	yes	2
GOLD-B004-12	openai	configuration_interaction	single_span	READY_FOR_OWNER_REVIEW	no	1
GOLD-B004-13	openai	exact_lookup	single_span	NEEDS_REPAIR	yes	2
GOLD-B004-14	openai	exact_lookup	multi_span	NEEDS_REPAIR	yes	2
GOLD-B004-15	openai	genuine_multi_hop	multi_document	NEEDS_REPAIR	yes	4

Findings by class

- **Scope in a heading** (06, 09, 14, 15) — the qualification a question relies on sat outside every span. Rule 2D of the generation brief forbids leaning on a header outside the exact evidence, so each anchor was extended or given a scope span.
- **Question broader than its evidence** (01, 07, 11) — "What happens when using server tools?" has many true answers; the span supports one.
- **Truncated critical strings** (06, 13) — 60-character cuts through a markdown link are not meaningful checkable strings.
- **Comparative anaphora** (10) — "a different `RealtimeModel`" is different from a default the span never named.
- **Taxonomy inflation** (01, 11) — a single conditional fact is not an interaction between settings; both relabelled to `error_behavior` with the reason recorded.
- **Not an ambiguity** (08) — recommended for rejection.
- **Noncritical flags left for the owner** (02, 05, 15) — surfaced, not overridden.

3. The multi-hop case, reviewed semantically

§3 of the review brief: the mechanical PASS is not to be trusted on its own. The composition check proves that neither span carries the other hop's critical strings. It says nothing about whether the two documents mean the same thing by `needs_approval`, or whether span 1 establishes the state span 2 tests. Those are the questions below.

A. Does span 1 establish the exact state tested by span 2?

Yes, and only after the repair.

Span 2 tests `needs_approval` being not `False`. Span 1 sets it to `True`. The state span 1 establishes is a member of the set span 2 tests, so the chain closes. What span 1 does not establish — and what the unrepaired candidate silently assumed — is that span 2's rule applies at all. That scope now sits inside span 2's anchor.

B. Does setting `needs_approval=True` satisfy '`needs_approval` is not `False`' without unsupported inference?

Yes.

`True` is not `False` is an identity, not an inference about the API. Span 1 also documents a second permitted value — an async callable — and span 2's rule covers that too, since a callable is likewise not `False`. The composed claim is stated for `True` only, which is the narrower and therefore safer reading.

C. Does span 1 come from the same SDK/tool concept as span 2?

Same SDK, different execution surface — and this was the defect.

Both spans are `openai-agents-python`. Span 1 is the general human-in-the-loop page; span 2 is the hosted multi-agent (experimental) surface, where the SDK's approval interruption mechanism is not available. The setting is the same setting; the surfaces are not the same surface. As generated, the question was unqualified, which made the composed answer false in the ordinary Runner flow — where `needs_approval = True` does exactly what span 1 says and pauses for approval. The repair puts 'hosted agents' into both the question and the anchor.

D. Are the two documents referring to compatible meanings of `needs_approval`?

Yes.

Both refer to the per-tool `needs_approval` setting on a function tool, with the same value domain (`True`, `False`, or a callable). Span 2's phrase 'any function tool whose `needs_approval` setting' matches span 1's subject exactly. This is not the `max_tokens` equivocation found among the near-miss pairs, where one span meant a request parameter and the other a `stop_reason` value.

E. Is neither span alone sufficient to answer the final question?

Yes — and span 1 alone is worse than insufficient.

A reader holding only span 1 concludes the run pauses for approval, which is the opposite of the outcome on this surface. A reader holding only span 2 knows tools are rejected but not that `True` is the value a person would have set or what they were trying to achieve. This is the property batch 003's four candidates lacked: there, either span answered on its own.

F. Does the composed answer follow directly?

Yes, once the scope is inside the evidence. No, as originally generated.

Repaired, the chain is: on the hosted-agent surface (span 2), a function tool whose `needs_approval` is not `False` is rejected before the request is sent; `True` is such a value (span 1); therefore a tool set to `True` is rejected there. Every step is stated in the evidence and nothing is assumed. Unrepaired, step one required the unstated assumption that the reader was on the hosted-agent surface, which is precisely the 'unstated compatibility assumption' §3 says to mark `NEEDS_REPAIR` for.

VERDICT

NEEDS_REPAIR, repaired; genuine_multi_hop preserved. Preserved: reasoning_type = genuine_multi_hop , evidence_shape = multi_document , requires_all_evidence = True .

WHAT THE MECHANICAL CHECK STILL CANNOT SEE

The composition check tests textual overlap between each hop's critical strings and the other span. It cannot detect equivocation (one string naming two things), it cannot detect a scope carried by a heading, and it cannot tell whether span 1's state is the state span 2 tests. All three had to be judged by reading. A reviewer who disagrees with any of them should reject this candidate rather than trusting the PASS.

4. Near-miss bridge pairs — the rule under test

5 pairs cleared every check in the composer except the entity-state rule: span 2's conditional must test the bridge entity's own state, not merely mention it while testing something else. Every one is judged a correct rejection, so on this evidence the rule is not too strict — it was the only check that caught them, since all 5 passed the composition check that is supposed to be the hostile one.

bridge entity	provider	span layout	verdict
OpenAIChatCompletionsModel	openai	two documents	CORRECT_REJECTION
allowed_callers	anthropic	same document	CORRECT_REJECTION
max_tokens	anthropic	two documents	CORRECT_REJECTION
tool_result	anthropic	two documents	CORRECT_REJECTION
view_range	anthropic	two documents	CORRECT_REJECTION

Diagnostic only. None of these is a batch-004 candidate, none was promoted, and the batch was not regenerated. Choosing candidates by re-reading the rejection list is how a benchmark ends up measuring its own generator.

`OpenAIChatCompletionsModel` · CORRECT_REJECTION

SPAN 1 — PROPOSED HOP 1

The built-in `OpenAIResponsesModel` and `OpenAIChatCompletionsModel` propagate an available server-generated `x-request-id` on their HTTP and SSE transport paths.

SPAN 2 — PROPOSED HOP 2

The OpenAI Chat Completions API can return audio output, but [`OpenAIChatCompletionsModel`] [`agents.models.openai_chatcompletions.OpenAIChatCompletionsModel`] does not currently convert audio output into Agents SDK run items.

WHY THE ENTITY-STATE RULE REJECTED IT

the span carries no conditional marker at all, so there is no clause that could test the entity's state

REVIEWER REASONING

Span 1 says the built-in models propagate `x-request-id` on their transport paths. Span 2 says the Chat Completions API can return audio output that the model does not convert into run items. These are two unrelated facts that happen to name the same class: request-id propagation has no bearing on audio conversion, and no answer follows from holding both. Span 2 contains no conditional at all, so there was nothing for span 1 to establish. The rejection is right, and it would have been right under any of the rules.

`allowed_callers` · CORRECT_REJECTION

SPAN 1 — PROPOSED HOP 1

Adding `allowed_callers: ["code_execution_20260120"]` to a tool definition is what makes that tool callable from within code execution (see [The `allowed_callers` field] (<https://platform.claude.com/docs/en/agents-and-tools/tool-use/programmatic-tool-calling#the-allowed-callers-field>)).

SPAN 2 — PROPOSED HOP 2

Including a code execution tool version in `allowed_callers` for such a tool causes the request to fail with a `400 invalid_request_error` whose message contains `Circular $ref detected`.

WHY THE ENTITY-STATE RULE REJECTED IT

the span's conditional markers are [`whose`], and none of their clauses contains `allowed_callers` — `whose` governs `message contains Circular $ref detected`.

REVIEWER REASONING

This is the closest of the five and still a correct rejection, though the rule caught it for a shallower reason than the one that matters. The rule saw that the only conditional marker in span 2 (`whose`) governs the error message rather than `allowed_callers`. The substantive problem is narrower: span 2's failure applies to a tool whose input schema contains a circular `$ref`, a qualification the phrase 'for such a tool' carries from text outside the span. Composing it with span 1 would produce 'adding `allowed_callers` makes the tool callable, and doing so fails with 400', which is false in general and true only for that unstated subcase. That is unsupported inference, which is the composition check's own §9 criterion. Span 1 also ends in a colon introducing a code block, so it is weak evidence on its own terms.

`max_tokens` · CORRECT REJECTION

SPAN 1 — PROPOSED HOP 1

```
* `budget_tokens` can exceed `max_tokens` here; the [budget rules]
(https://platform.claude.com/docs/en/build-with-claude/extended-thinking#budget-rules-and-tuning) explain
this exception.
* Interleaved thinking is only supported for [tools used through the Messages API]
(https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview).
```

SPAN 2 — PROPOSED HOP 2

```
The loop exits on any other stop reason (`end_turn`, `max_tokens`, `stop_sequence`, or `refusal`),
which means Claude has either produced a final answer or stopped for another reason that your application
should handle.
```

WHY THE ENTITY-STATE RULE REJECTED IT

the span carries no conditional marker at all, so there is no clause that could test the entity's state

REVIEWER REASONING

The most instructive of the five. Span 1 uses `max_tokens` as a request parameter that `budget_tokens` may exceed during interleaved thinking. Span 2 uses `"max_tokens"` as one of the `stop_reason` values on which the agentic loop exits. The bridge test matched a string that names two different things in the two spans, so there was never an entity to chain through. This is equivocation, not a hop, and it is worth recording that the bridge-entity requirement — the entity must appear in both spans — cannot detect it.

`tool_result` · CORRECT REJECTION

SPAN 1 — PROPOSED HOP 1

```
**Text-only tool result content:** The `content` of each `tool_result` that answers a programmatic call
must be a string or `text` blocks.
```

SPAN 2 — PROPOSED HOP 2

```
Adding anything after the `tool_result` blocks in that user message, such as text, ends the assistant
turn; for a server tool Claude called directly, the request then fails with a 400 `invalid_request_error`
that names the unresolved server tool:
```

WHY THE ENTITY-STATE RULE REJECTED IT

the span carries no conditional marker at all, so there is no clause that could test the entity's state

REVIEWER REASONING

Span 1 requires the `content` of a `tool_result` answering a programmatic call to be a string or text blocks. Span 2 says that adding anything after the `tool_result` blocks in a user message ends the assistant turn, and fails with 400 for a server tool Claude called directly. Both are formatting rules about `tool_result` and neither depends on the other: satisfying span 1 tells a reader nothing about span 2's outcome. Two rules in one subject area are still two lookups. Span 2 also ends in a colon, and its condition is about message structure rather than about the entity's configured state.

`view_range` · CORRECT REJECTION

SPAN 1 — PROPOSED HOP 1

- * ``command``: Must be "view"
- * ``path``: The path to the file or directory to view
- * ``view_range`` (optional): An array of two integers specifying the start and end line numbers to view.

SPAN 2 — PROPOSED HOP 2

``view_range`` is optional and applies to text-file views: ``[start_line, end_line]`` returns those lines, and ``[start_line, -1]`` returns everything from ``start_line`` to the end of the file.

WHY THE ENTITY-STATE RULE REJECTED IT

the span carries no conditional marker at all, so there is no clause that could test the entity's state

REVIEWER REASONING

Span 1 is a bullet list describing `view_range` in the text editor tool; span 2 describes `view_range` in the memory tool. They are two parallel definitions of a parameter in two different tools, which is the plainest possible case of multi-span-is-not-multi-hop. Span 1 is also a list fragment whose stem sits outside the span, so it would have failed the evidence-scope check even if the pair had been a chain.

5. Erratum — the near-miss count

The batch-004 results PDF said **three** near-miss pairs. The correct number is **5**. The three came from a manual probe run mid-development, with the composer's per-run limit and its used-fact set in force, before the entity-state rule existed. The diagnostic now derives the set properly and the PDF builder reads the count from it instead of hardcoding it.

The generation report's own figures were computed from the run and are unchanged: **559** pairs tested, **1** passed, **558** rejected. No candidate changed status because of the erratum.

One finding from the corrected diagnostic is a gap in a check rather than a judgement about a pair. The `max_tokens` pair failed because span 1 means a request parameter and span 2 means a `stop_reason` value. The bridge requirement — the entity must appear in both spans — is a string test and cannot see equivocation. Nothing in the current composer can. That belongs in batch 005's design, preregistered before it sees a candidate.

6. How a repair is gated

Every anchor repair is a strict outward growth of the span it replaces; an anchor that moves rather than grows is a different claim wearing the same candidate id. Both hashes are recorded. A repaired candidate is approved by quoting the post-repair evidence hash, and the importer refuses an approval quoting the pre-repair hash — so a repair cannot be waved through by someone who only ever saw the original.

The generation artifact is not rewritten. `gold_review_batch_004.json` is byte-identical to what the generator produced; repairs live beside it in `gold_review_batch_004_repairs.json` with the original offsets, text and hashes preserved.

7. The fifteen candidates

Each card carries the final question, answer and claims, the exact evidence with its hash, what the review found, and what it changed. This is the material to check.

GOLD-B004-01 · NEEDS_REPAIR

anthropic · Stop reasons and fallback · Best practices for handling stop reasons › Implement retry logic for pause_turn
reasoning type error_behavior (generated as configuration_interaction) · shape single_span · requires all evidence False ·
precheck holdout-ready True

Q. When using server tools, what does the API return if the server-side sampling loop reaches its iteration limit?

A. The API may return pause_turn .

ATOMIC CLAIMS

1. When using [server tools](https://platform.claude.com/docs/en/agents-and-tools/tool-use/server-tools), the API may return pause_turn if the server-side sampling loop reaches its iteration limit (default 10).

EXACT EVIDENCE

E1 · VER_4D14AEC24504F4B8F6F28938B84587DC 80272-80481 (209 CHARS) · BEST PRACTICES FOR HANDLING STOP REASONS › IMPLEMENT RETRY LOGIC FOR PAUSE_TURN

When using [server tools](https://platform.claude.com/docs/en/agents-and-tools/tool-use/server-tools), the API may return `pause\_turn` if the server-side sampling loop reaches its iteration limit (default 10).

critical strings: pause_turn
evidence_hash b13da047087150f9eefa746b904f26b9fc06d87cbc58b912916dd6eba0a74231

INTERNAL REVIEW FINDINGS

- QUESTION_SCOPE: 'What happens when using server tools?' is far broader than its evidence. The span answers one conditional fact — what the API may return when the server-side sampling loop reaches its iteration limit — and a reader could give a dozen true answers to the question as asked.
- CATEGORY: the source states one direct conditional fact, not an interaction between two settings. §8 says relabel rather than inflate reasoning complexity.

REPAIRS MADE

- **question rewritten**
was: What happens when using server tools?
now: When using server tools, what does the API return if the server-side sampling loop reaches its iteration limit?
- **answer rewritten**
was: The API may return pause_turn if the server-side sampling loop reaches its iteration limit (default 10).
now: The API may return pause_turn .
- **reasoning_type rewritten**
was: configuration_interaction
now: error_behavior

GOLD-B004-02 · READY_FOR_OWNER_REVIEW

anthropic · Web search tool · Response › `pause\_turn` stop reason
reasoning type `configuration_interaction` · shape `single_span` · requires all evidence False · precheck holdout-ready True

Q. What happens if Claude calls web search and one of your client tools in the same group of parallel tool calls?

A. The API returns `stop_reason: "tool_use"` instead and does not run the search yet.

ATOMIC CLAIMS

1. If Claude calls web search and one of your client tools in the same group of parallel tool calls, the API returns `stop_reason: "tool_use"` instead and does not run the search yet.

EXACT EVIDENCE

E1 · VER_53DA2F78E855C75EC755089C13D44C28 22695–22875 (180 CHARS) · RESPONSE › `PAUSE\_TURN` STOP REASON

If Claude calls web search and one of your client tools in the same group of parallel tool calls, the API returns `stop_reason: "tool_use"` instead and does not run the search yet.

critical strings: `stop_reason`, `tool_use`

evidence_hash `0f83094654abd858b9e46cff62a781ffc8b0bb12c679beb91ed3498045109e06`

INTERNAL REVIEW FINDINGS

- **NONCRITICAL_ANAPHORA:** the span says the API returns `tool_use` 'instead', and what it is instead of — the `pause_turn` behaviour — sits in the preceding paragraph, outside the span. The fact is answerable without resolving it, so this is noncritical, but §2B requires an explicit human override rather than a silent pass.

GOLD-B004-03 · READY_FOR_OWNER_REVIEW

anthropic · Claude API errors · Common validation errors › Thinking blocks cannot be modified
reasoning type `error_behavior` · shape `single_span` · requires all evidence False · precheck holdout-ready True

Q. What happens if the most recent assistant message contains `thinking` or `redacted_thinking` blocks that were edited, reordered, filtered out, or reconstructed before being sent back to the API?

A. The request returns a 400 `invalid_request_error`.

ATOMIC CLAIMS

1. If the most recent assistant message contains `thinking` or `redacted_thinking` blocks that were edited, reordered, filtered out, or reconstructed before being sent back to the API, the request returns a 400 `invalid_request_error`.

EXACT EVIDENCE

E1 · VER_0774CA0093FF4A846753577C9A4A39D5 19838–20070 (232 CHARS) · COMMON VALIDATION ERRORS › THINKING BLOCKS CANNOT BE MODIFIED

If the most recent assistant message contains `thinking` or `redacted_thinking` blocks that were edited, reordered, filtered out, or reconstructed before being sent back to the API, the request returns a 400 `invalid_request_error`.

critical strings: `thinking`, `redacted_thinking`, `invalid_request_error`

evidence_hash `1f1dbc851bb4029f1d827d1c11462573659eaa43a6e23acae8e9b1e0a916cdff`

INTERNAL REVIEW FINDINGS

- No finding. The candidate stands as generated.

GOLD-B004-04 · NEEDS_REPAIR

anthropic · Thinking · Thinking and the context window

reasoning type `error_behavior` · shape `single_span` · requires all evidence False · precheck holdout-ready True

Q. On Claude 4.5 models and newer, if input tokens plus `max_tokens` exceed the context window size, what happens when generation reaches the context window limit?

A. The API accepts the request, and generation stops with `stop_reason: "model_context_window_exceeded"` instead of returning an error.

ATOMIC CLAIMS

1. On Claude 4.5 models and newer, if input tokens plus `max_tokens` exceeds the context window size, the API accepts the request.
2. If generation then reaches the context window limit, it stops with `stop_reason: "model_context_window_exceeded"` instead of returning an error.

EXACT EVIDENCE

E1 · VER_012B734775E7EDB2649D3A9DDFD93070 47953–48225 (272 CHARS) · THINKING AND THE CONTEXT WINDOW

On Claude 4.5 models and newer, if input tokens plus `max_tokens` exceeds the context window size, the API accepts the request. If generation then reaches the context window limit, it stops with `stop_reason: "model_context_window_exceeded"` instead of returning an error.

critical strings: `Claude 4.5 models and newer`, `max_tokens`, `stop_reason`, `model_context_window_exceeded`
evidence_hash `8eb88b1c77cc0a04bd7ffe36232db6fb450a7bf9b650087984be147984df7753`

INTERNAL REVIEW FINDINGS

- **CRITICAL_ANAPHORA:** 'If generation then reaches the context window limit' — 'then' refers to the preceding sentence, which is outside the span.
- **MODEL_SCOPE:** the antecedent is 'On Claude 4.5 models and newer, if input tokens plus `max_tokens` exceeds the context window size, the API accepts the request.' The very next sentence in the source says earlier models return a validation error instead, so the claim as anchored over-generalises across models.

REPAIRS MADE

- **E1 anchor extended** (evidence_boundary_completion)

was `48081–48225`, hash `13f22cd01eeaeaad4e7c51e02e5ea5347541780f320e43c4cb816a141c933afb`
now `47953–48225`, hash `8eb88b1c77cc0a04bd7ffe36232db6fb450a7bf9b650087984be147984df7753`

- **question rewritten**

was: What happens if generation then reaches the context window limit?

now: On Claude 4.5 models and newer, if input tokens plus `max_tokens` exceed the context window size, what happens when generation reaches the context window limit?

- **answer rewritten**

was: It stops with `stop_reason: "model_context_window_exceeded"` instead of returning an error.

now: The API accepts the request, and generation stops with `stop_reason: "model_context_window_exceeded"` instead of returning an error.

- **atomic_claims rewritten**

was: ['If generation then reaches the context window limit, it stops with `stop_reason: "model_context_window_exceeded"` instead of returning an error.']

now: ['On Claude 4.5 models and newer, if input tokens plus `max_tokens` exceeds the context window size, the API accepts the request.', 'If generation then reaches the context window limit, it stops with `stop_reason: "model_context_window_exceeded"` instead of returning an error.']

GOLD-B004-05 · READY_FOR_OWNER_REVIEW

anthropic · Web search tool · Tool definition › Max uses
reasoning type error_behavior · shape single_span · requires all evidence False · precheck holdout-ready True

Q. What happens if Claude attempts more searches than allowed?

A. The web_search_tool_result is an error with the max_uses_exceeded error code.

ATOMIC CLAIMS

- 1. If Claude attempts more searches than allowed, the web_search_tool_result is an error with the max_uses_exceeded error code.

EXACT EVIDENCE

E1 · VER_53DA2F78E855C75EC755089C13D44C28 15475–15603 (128 CHARS) · TOOL DEFINITION › MAX USES

If Claude attempts more searches than allowed, the `web\_search\_tool\_result` is an error with the `max\_uses\_exceeded` error code.

critical strings: web_search_tool_result , max_uses_exceeded
evidence_hash 9d881c94127cc6d29cf87e6cce6611ee77ac7b4fdbfb19477653a4f5b0a8fa08

INTERNAL REVIEW FINDINGS

- NONCRITICAL_DEPENDENCY: 'more searches than allowed' does not name the parameter that sets the limit (max_uses), which is defined in the preceding sentence. The question is answerable without it, so the anchor stands; an owner who wants the parameter named should choose NEEDS_EDIT and the span can be extended backwards by 60 characters.

GOLD-B004-06 · NEEDS_REPAIR

anthropic · Web search tool · Tool definition › Localization

reasoning type `exact_lookup` · shape `single_span` · requires all evidence False · precheck holdout-ready True

Q. Within the `user_location` parameter, what value does the `timezone` field take?

A. The IANA timezone ID.

ATOMIC CLAIMS

1. The `user_location` parameter allows you to localize search results based on a user's location.
2. `timezone` : The [IANA timezone ID](https://en.wikipedia.org/wiki/List_of_tz_database_time_zones).

EXACT EVIDENCE

E1 · VER_53DA2F78E855C75EC755089C13D44C28 16295-16843 (548 CHARS) · TOOL DEFINITION › LOCALIZATION

The ``user_location`` parameter allows you to localize search results based on a user's location. Provide at least one of ``city``, ``region``, ``country``, or ``timezone``.

```
* `type`: The type of location (must be `approximate`)  
* `city`: The city name  
* `region`: The region or state  
* `country`: The two-letter [ISO 3166-1 alpha-2](https://en.wikipedia.org/wiki/ISO_3166-1_alpha-2) country code. The API rejects unsupported country codes with a 400 error.  
* `timezone`: The [IANA timezone ID](https://en.wikipedia.org/wiki/List_of_tz_database_time_zones).
```

critical strings: `user_location`, `timezone`, IANA timezone ID

evidence_hash b38e23b266e3c424991cafafdc60be26959220d2a50a395e7ae813d5fba08c7c

INTERNAL REVIEW FINDINGS

- CLAIM_SCOPE: the span is a bare definition bullet. The parent that gives `timezone` its meaning — the `user_location` parameter — is in the section heading and the list stem, both outside the span. §2D forbids relying on a header outside the exact evidence.
- GENERIC_IDENTIFIER: 'What is the `timezone` option?' names an identifier that exists in many APIs (§6).
- CRITICAL_STRING: one critical string was a 60-character truncation of a markdown link — 'The [IANA timezone ID] (https://en.wikipedia.org/wiki/List_of' — which is not a meaningful checkable string.

REPAIRS MADE

- **E1 anchor extended** (evidence_scope_completion)

was 16744-16843, hash 0bd723f53fa5e63b4e74a9bd5ffb4a3886ed049776ce3c69528c18e1a7371371

now 16295-16843, hash b38e23b266e3c424991cafafdc60be26959220d2a50a395e7ae813d5fba08c7c

- **question rewritten**

was: What is the `timezone` option?

now: Within the `user_location` parameter, what value does the `timezone` field take?

- **answer rewritten**

was: The [IANA timezone ID](https://en.wikipedia.org/wiki/List_of_tz_database_time_zones).

now: The IANA timezone ID.

- **atomic_claims rewritten**

was: [' `timezone` : The [IANA timezone ID](https://en.wikipedia.org/wiki/List_of_tz_database_time_zones).']

now: ['The `user_location` parameter allows you to localize search results based on a user's location.', ' `timezone` : The [IANA timezone ID](https://en.wikipedia.org/wiki/List_of_tz_database_time_zones).']

GOLD-B004-07 · NEEDS_REPAIR

anthropic · Prompting best practices · Thinking and reasoning › Overthinking and excessive thoroughness
reasoning type lifecycle_compatibility_migration · shape single_span · requires all evidence False · precheck holdout-ready True

Q. If I need a hard ceiling on thinking costs, what is the support status of extended thinking with a budget_tokens cap on Opus 4.6 and Sonnet 4.6?

A. It is still functional but deprecated.

ATOMIC CLAIMS

1. If you need a hard ceiling on thinking costs, extended thinking with a budget_tokens cap is still functional on Opus 4.6 and Sonnet 4.6 but is deprecated.

EXACT EVIDENCE

E1 · VER_0A5B292D4854B92DB0A9E025B4949123 27270–27426 (156 CHARS) · THINKING AND REASONING › OVERTHINKING AND EXCESSIVE THOROUGHNESS

If you need a hard ceiling on thinking costs, extended thinking with a `budget\_tokens` cap is still functional on Opus 4.6 and Sonnet 4.6 but is deprecated.

critical strings: budget_tokens , Opus 4.6 , deprecated

evidence_hash 6defce92772b6eefb4f7e6e8c08c6c7107cc017e934290819360eb5bfd38c874

INTERNAL REVIEW FINDINGS

- QUESTION_FORM: 'What happens if you need a hard ceiling on thinking costs?' asks what happens when a person has a requirement, which is not a behaviour the documentation can answer. The evidence states a support status, so the question should ask for one.

REPAIRS MADE

- **question rewritten**

was: What happens if you need a hard ceiling on thinking costs?

now: If I need a hard ceiling on thinking costs, what is the support status of extended thinking with a budget_tokens cap on Opus 4.6 and Sonnet 4.6?

- **answer rewritten**

was: Extended thinking with a budget_tokens cap is still functional on Opus 4.6 and Sonnet 4.6 but is deprecated.

now: It is still functional but deprecated.

GOLD-B004-08 · REJECT_RECOMMENDED

openai · Structured Outputs Parsing Helpers · Streaming Helpers › Chat Completions API › Chat Completions Events › ContentDeltaEvent
reasoning type `ambiguity_disambiguation` · shape `multi_span` · requires all evidence True · precheck holdout-ready True

Q. In a `ContentDeltaEvent`, what does the `type` field contain, and how does that differ from `ContentDoneEvent`?

A. In `ContentDeltaEvent`, `type` is: `"content.delta"`. In `ContentDoneEvent`, `type` is: `"content.done"`.

ATOMIC CLAIMS

1. In `ContentDeltaEvent`, `type` is: `"content.delta"`.
2. In `ContentDoneEvent`, `type` is: `"content.done"`.

EXACT EVIDENCE

E1 · VER_57E26A49B0A3714F3E90376D014D7F52 5814–5841 (27 CHARS) · STREAMING HELPERS › CHAT COMPLETIONS API › CHAT COMPLETIONS EVENTS › CONTENTDELTAEVENT

```
- `type`: `"content.delta"`
```

critical strings: `type`, `"content.delta"`

evidence_hash 9d90d13b5ef5041ad8d20a4c0acf1a56b5845947c2b74462a46d2700c7fa393d

E2 · VER_57E26A49B0A3714F3E90376D014D7F52 6134–6160 (26 CHARS) · STREAMING HELPERS › CHAT COMPLETIONS API › CHAT COMPLETIONS EVENTS › CONTENTDONEEVENT

```
- `type`: `"content.done"`
```

critical strings: `type`, `"content.done"`

evidence_hash c4e5d438b90d3496919b5293c18370f6d3ed060409e90fce345e3b8ac4a76b12

INTERNAL REVIEW FINDINGS

- NOT_AMBIGUITY: `type` on `ContentDeltaEvent` and `ContentDoneEvent` is a discriminator constant. A tagged union whose tag differs per member is not a case where a developer must select a scope to resolve a meaning — it is two literal lookups, and §7 says to relabel rather than keep the label to hit a category target.
- CLAIM_SCOPE: both spans are 27 and 26 characters and contain neither event-type name, so the question's scope lives entirely in headings outside the evidence (§2D). This part is repairable — expanding each span to its `#### EventName` heading costs only ~100 characters — but repairing the scope does not make the case a disambiguation.
- CEILING: relabelling to `exact_lookup` would take that category to 4 against the §5 maximum of 3.

GOLD-B004-09 · NEEDS_REPAIR

openai · Structured Outputs Parsing Helpers · Streaming Helpers · Chat Completions API · Chat Completions Events ·

FunctionToolCallArgumentsDeltaEvent

reasoning type `ambiguity_disambiguation` · shape `multi_span` · requires all evidence True · precheck holdout-ready True

Q. In a `FunctionToolCallArgumentsDeltaEvent`, what does the `parsed_arguments` field contain, and how does that differ from `FunctionToolCallArgumentsDoneEvent`?

A. In `FunctionToolCallArgumentsDeltaEvent`, `parsed_arguments` is: The partially parsed arguments object. In `FunctionToolCallArgumentsDoneEvent`, `parsed_arguments` is: The fully parsed arguments object. If you used `openai.pydantic_function_tool()` this will be an instance of the given model.

ATOMIC CLAIMS

1. In `FunctionToolCallArgumentsDeltaEvent`, `parsed_arguments` is: The partially parsed arguments object.
2. In `FunctionToolCallArgumentsDoneEvent`, `parsed_arguments` is: The fully parsed arguments object. If you used `openai.pydantic_function_tool()` this will be an instance of the given model.

EXACT EVIDENCE

E1 · VER_57E26A49B0A3714F3E90376D014D7F52 6627–6997 (370 CHARS) · STREAMING HELPERS · CHAT COMPLETIONS API · CHAT COMPLETIONS EVENTS · FUNCTIONTOOLCALLARGUMENTSDELTAEVENT

```
#### FunctionToolCallArgumentsDeltaEvent
```

Emitted when a chunk contains part of a function tool call's arguments.

```
- `type`: "tool_calls.function.arguments.delta"
- `name`: The name of the function being called
- `index`: The index of the tool call
- `arguments`: The accumulated raw JSON string of arguments
- `parsed_arguments`: The partially parsed arguments object
```

critical strings: `FunctionToolCallArgumentsDeltaEvent`, `parsed_arguments`, The partially parsed arguments object
evidence_hash 53b4b866ce9f1066a9bbe80d16dacdd21318dd256b8564216256943b8adec57e

E2 · VER_57E26A49B0A3714F3E90376D014D7F52 7072–7509 (437 CHARS) · STREAMING HELPERS · CHAT COMPLETIONS API · CHAT COMPLETIONS EVENTS · FUNCTIONTOOLCALLARGUMENTSDONEEVENT

```
#### FunctionToolCallArgumentsDoneEvent
```

Emitted when a function tool call's arguments are complete.

```
- `type`: "tool_calls.function.arguments.done"
- `name`: The name of the function being called
- `index`: The index of the tool call
- `arguments`: The full raw JSON string of arguments
- `parsed_arguments`: The fully parsed arguments object. If you used `openai.pydantic_function_tool()` this will be an instance of the given model.
```

critical strings: `FunctionToolCallArgumentsDoneEvent`, `parsed_arguments`, The fully parsed arguments object
evidence_hash 2c4842349ec2c5fd3374e46a6cd2378745ff41ad0f8ac74ce9b24f259dd8aa26

INTERNAL REVIEW FINDINGS

- **AMBIGUITY_CONFIRMED:** `parsed_arguments` genuinely means different things on the two events — a partially parsed object mid-stream, a fully parsed object (a pydantic model instance where `openai.pydantic_function_tool()` was used) when complete. A developer holding one event and reading the other's documentation would be wrong, which is the realistic confusion §11 asks for.
- **CLAIM_SCOPE:** as anchored, neither span contains its event-type name, so the scope the question names is in a heading outside the evidence (§2D).

REPAIRS MADE

- **E1 anchor extended (evidence_scope_completion)**

was 6938-6997, hash a3608232ce720258f2e6af35dce62d3ec8968dd22a187707c40130688f78032b

now 6627-6997, hash 53b4b866ce9f1066a9bbe80d16dacdd21318dd256b8564216256943b8adec57e

- **E2 anchor extended (evidence_scope_completion)**

was 7362-7509, hash 24e776c42e7a5e4cecfb38c19c02bf8757d107244a0b584f4701adf6c1231f5f

now 7072-7509, hash 2c4842349ec2c5fd3374e46a6cd2378745ff41ad0f8ac74ce9b24f259dd8aa26

GOLD-B004-10 · NEEDS_REPAIR

openai · Realtime agents guide · Realtime agents guide > Session lifecycle
reasoning type `configuration_interaction` · shape `single_span` · requires all evidence False · precheck holdout-ready True

Q. `RealtimeRunner` uses `OpenAIRealtimeWebSocketModel` by default. What happens if I pass a different `RealtimeModel` ?

A. The same session lifecycle and agent features still apply, while the connection mechanics can change.

ATOMIC CLAIMS

1. By default, `RealtimeRunner` uses `OpenAIRealtimeWebSocketModel`, so the default Python path is a server-side WebSocket connection to the Realtime API.
2. If you pass a different `RealtimeModel`, the same session lifecycle and agent features still apply, while the connection mechanics can change.

EXACT EVIDENCE

E1 · VER_14A2187CF2216B9D56C213B520A28479 1827-2121 (294 CHARS) · REALTIME AGENTS GUIDE > SESSION LIFECYCLE

By default, `RealtimeRunner` uses `OpenAIRealtimeWebSocketModel`, so the default Python path is a server-side WebSocket connection to the Realtime API. If you pass a different `RealtimeModel`, the same session lifecycle and agent features still apply, while the connection mechanics can change.

critical strings: `RealtimeRunner`, `OpenAIRealtimeWebSocketModel`, `RealtimeModel`
evidence_hash 352fcab03e4bb99e38aeb11550323d6e3efe49a101b144bd2ea2a2f28ac35d9d

INTERNAL REVIEW FINDINGS

- **COMPARATIVE_ANAPHORA:** 'a different `RealtimeModel`' is different from a default the span does not name. The default — `OpenAIRealtimeWebSocketModel` — is established in the preceding sentence, outside the anchor.
- **CATEGORY:** with the default inside the anchor this is a real configuration interaction (which transport model is used changes connection mechanics while leaving the session lifecycle alone), so the label stands once the scope does.

REPAIRS MADE

- **E1 anchor extended** (evidence_boundary_completion)

was 1979-2121, hash ad1950090760f8b3b9378e1d9d7cddb5d66e5921d1a9005eda690004597ed6f5
now 1827-2121, hash 352fcab03e4bb99e38aeb11550323d6e3efe49a101b144bd2ea2a2f28ac35d9d

- **question rewritten**

was: What happens if you pass a different `RealtimeModel` ?
now: `RealtimeRunner` uses `OpenAIRealtimeWebSocketModel` by default. What happens if I pass a different `RealtimeModel` ?

- **atomic_claims rewritten**

was: ['If you pass a different `RealtimeModel`, the same session lifecycle and agent features still apply, while the connection mechanics can change.']
now: ['By default, `RealtimeRunner` uses `OpenAIRealtimeWebSocketModel`, so the default Python path is a server-side WebSocket connection to the Realtime API.', 'If you pass a different `RealtimeModel`, the same session lifecycle and agent features still apply, while the connection mechanics can change.']

GOLD-B004-11 · NEEDS REPAIR

openai · Streaming · Streaming › Streaming and approvals

reasoning type `error_behavior` (generated as `configuration_interaction`) · shape `single_span` · requires all evidence False · precheck holdout-ready True

Q. In a streamed run, what happens if a tool requires approval?

A. `result.stream_events()` finishes and pending approvals are exposed in `RunResultStreaming.interruptions`.

ATOMIC CLAIMS

1. If a tool requires approval, `result.stream_events()` finishes and pending approvals are exposed in `[RunResultStreaming.interruptions][agents.result.RunResultStreaming.interruptions]`.

EXACT EVIDENCE

E1 · VER_12004469F7A5592CD1E6CAB936117FCE 2472–2657 (185 CHARS) · STREAMING › STREAMING AND APPROVALS

```
If a tool requires approval, `result.stream_events()` finishes and pending approvals are exposed in
[ `RunResultStreaming.interruptions` ][agents.result.RunResultStreaming.interruptions].
```

critical strings: `stream_events`, `RunResultStreaming.interruptions`

evidence_hash `fd6ec23ac1830d0625b7509d4ef3572a6ec90090565db15c3d8402acbc147908`

INTERNAL REVIEW FINDINGS

- **QUESTION_SCOPE:** 'What happens if a tool requires approval?' is generic across SDKs and run modes. The evidence is about a streamed run — it names `result.stream_events()` — so the question can carry that scope without adding anything the span does not say.
- **CATEGORY:** one direct conditional fact about run behaviour, not an interaction between two settings (§8).

REPAIRS MADE

- **question rewritten**
was: What happens if a tool requires approval?
now: In a streamed run, what happens if a tool requires approval?
- **reasoning_type rewritten**
was: `configuration_interaction`
now: `error_behavior`

GOLD-B004-12 · READY_FOR_OWNER_REVIEW

openai · Streaming · Streaming › Cancel streaming after the current turn

reasoning type `configuration_interaction` · shape `single_span` · requires all evidence False · precheck holdout-ready True

Q. What happens if you are manually continuing from `result.to_input_list(mode="normalized")`, and `cancel(mode="after_turn")` stops after a tool turn?

A. Rerun `result.last_agent` with that normalized input to continue the unfinished existing user turn instead of appending a fresh user turn right away.

ATOMIC CLAIMS

1. If you are manually continuing from `[ result.to_input_list(mode="normalized") ]` `[agents.result.RunResultBase.to_input_list]`, and `cancel(mode="after_turn")` stops after a tool turn, rerun `result.last_agent` with that normalized input to continue the unfinished existing user turn instead of appending a fresh user turn right away.

EXACT EVIDENCE

E1 · VER_12004469F7A5592CD1E6CAB936117FCE 3845–4175 (330 CHARS) · STREAMING › CANCEL STREAMING AFTER THE CURRENT TURN

```
If you are manually continuing from [`result.to_input_list(mode="normalized")`]
[agents.result.RunResultBase.to_input_list], and `cancel(mode="after_turn")` stops after a tool turn,
rerun `result.last_agent` with that normalized input to continue the unfinished existing user turn
instead of appending a fresh user turn right away.
```

critical strings: `to_input_list`, `after_turn`, `result.last_agent`

evidence_hash `cfb7414fc8aeb9482635a45eede6ce305286f8df5f58cf54cc0818e14e954232`

INTERNAL REVIEW FINDINGS

- The question is long, because the source's condition has two parts and both matter. Shortening it would drop a condition the answer depends on, so it stands as written.

GOLD-B004-13 · NEEDS_REPAIR

openai · Handoffs · (1)! › Customizing handoffs via the `handoff()` function
reasoning type `exact_lookup` · shape `single_span` · requires all evidence False · precheck holdout-ready True

Q. For a handoff, what does the `input_type` option specify?

A. The schema for the handoff tool-call arguments. When set, the parsed payload is passed to `on_handoff`.

ATOMIC CLAIMS

1. `input_type`: The schema for the handoff tool-call arguments. When set, the parsed payload is passed to `on_handoff`.

EXACT EVIDENCE

E1 · VER_1C77F33B04FFFFFA285EA7E61C2A89653 2332–2453 (121 CHARS) · (1)! › CUSTOMIZING HANDOFFS VIA THE `HANDOFF()` FUNCTION

```
- `input_type`: The schema for the handoff tool-call arguments. When set, the parsed payload is passed to `on_handoff`.
```

critical strings: `input_type`, The schema for the handoff tool-call arguments, `on_handoff`
evidence_hash 3b8a83ecd785ac37429a82c6060abc61a4f6d4ac14a89586830b25a4ec75f25e

INTERNAL REVIEW FINDINGS

- GENERIC_IDENTIFIER: 'What is the `input_type` option?' does not say what it is an option of (§6). Unlike B004-06 and B004-14 the span itself says 'the handoff tool-call arguments', so the scope can be added to the question without touching the evidence.
- CRITICAL_STRING: one critical string was a 60-character truncation — 'The schema for the handoff tool-call arguments. When set, th'.

REPAIRS MADE

- **question rewritten**
was: What is the `input_type` option?
now: For a handoff, what does the `input_type` option specify?

GOLD-B004-14 · NEEDS_REPAIR

openai · Handoffs · (1)! › Customizing handoffs via the `handoff()` function
reasoning type `exact_lookup` · shape `multi_span` · requires all evidence True · precheck holdout-ready True

Q. In the `handoff()` function, what does the `input_filter` option do?

A. It filters the input received by the next agent.

ATOMIC CLAIMS

1. The `[handoff()][agents.handoffs.handoff]` function lets you customize things.
2. `input_filter` : This lets you filter the input received by the next agent.

EXACT EVIDENCE

E1 · VER_1C77F33B04FFFFFA285EA7E61C2A89653 1576–1654 (78 CHARS) · (1)! › CUSTOMIZING HANDOFFS VIA THE `HANDOFF()` FUNCTION

The `[handoff()][agents.handoffs.handoff]` function lets you customize things.

critical strings: `handoff()`
evidence_hash 6ed28e39bdd7b060856374798eb975198a88cf4e9866ee4f8ccd25450138caba

E2 · VER_1C77F33B04FFFFFA285EA7E61C2A89653 2454–2552 (98 CHARS) · (1)! › CUSTOMIZING HANDOFFS VIA THE `HANDOFF()` FUNCTION

- `input_filter`: This lets you filter the input received by the next agent. See below for more.

critical strings: `input_filter`, This lets you filter the input received by the next agent
evidence_hash 1c50ad6eed23afe1fd9cb3c931d490f140d744f683d5f95de266846dae0364a

INTERNAL REVIEW FINDINGS

- CLAIM_SCOPE: the span reads '`input_filter`: This lets you filter the input received by the next agent.' and contains no mention of handoffs. The scope is entirely in the section heading and the list stem, both outside the evidence (§2D).
- REPAIR_SHAPE: a contiguous expansion back to the list stem would swallow five unrelated field definitions and produce a 976-character anchor. §16 asks for precise spans, so the stem is added as its own span instead.

REPAIRS MADE

- **E0 scope span added** (evidence_scope_completion)

1576–1654, hash 6ed28e39bdd7b060856374798eb975198a88cf4e9866ee4f8ccd25450138caba

- **question rewritten**

was: What is the `input_filter` option?

now: In the `handoff()` function, what does the `input_filter` option do?

- **answer rewritten**

was: This lets you filter the input received by the next agent.

now: It filters the input received by the next agent.

- **atomic_claims rewritten**

was: ['`input_filter` : This lets you filter the input received by the next agent.']

now: ['The `[handoff()][agents.handoffs.handoff]` function lets you customize things.', '`input_filter` : This lets you filter the input received by the next agent.']

GOLD-B004-15 · NEEDS_REPAIR

openai · Human-in-the-loop · Human-in-the-loop > Marking tools that need approval

reasoning type `genuine_multi_hop` · shape `multi_document` · requires all evidence `True` · precheck holdout-ready `True`

Q. If I set `needs_approval` to `True` on a function tool used with hosted agents, what happens?

A. The tool requires approval, but SDK tool-approval interruptions are not supported there, so a function tool whose `needs_approval` is not `False` is rejected before the request is sent.

ATOMIC CLAIMS

1. Set `needs_approval` to `True` to always require approval or provide an async function that decides per call.
2. SDK tool approval interruptions are not supported: any function tool whose `needs_approval` setting is not `False` is rejected before the request is sent.

Composed claim. For hosted agents, a function tool with `needs_approval` set to `True` has a value that is not `False`, so it is rejected before the request is sent rather than pausing for approval.

EXACT EVIDENCE

E1 · VER_AE3BFCC42C733C5051ABDA30F0F6DB07 1327–1436 (109 CHARS) · HUMAN-IN-THE-LOOP > MARKING TOOLS THAT NEED APPROVAL

Set ``needs_approval`` to ``True`` to always require approval or provide an async function that decides per call.

critical strings: `needs_approval`, `True`

evidence_hash 72cb1e104a68797ed1296cfed17b04ba519cd1890449a2e5542f4ed36ad4e836

E2 · VER_AE909BF8B4BBBE1D1A11119447F7AC94 16313–16910 (597 CHARS) · MODELS > OPENAI MODELS > HOSTED MULTI-AGENT (EXPERIMENTAL) > LOCAL FUNCTION TOOLS

All hosted agents share the model and tools configured for the request. The Responses API decides which hosted agent calls a function. The normal SDK Runner executes the function locally and injects a ``function_call_output`` with the same call ID into the active WebSocket response, which lets the service resume the original hosted caller. Function execution still passes through the Runner's normal guardrails, hooks, and failure conversion. SDK tool approval interruptions are not supported: any function tool whose ``needs_approval`` setting is not ``False`` is rejected before the request is sent.

critical strings: `hosted agents`, `needs_approval`, `False`

evidence_hash b0ef211a15b2f158ac09382986237d5050567eaf25439c000f7fc76a3415bafd

INTERNAL REVIEW FINDINGS

- **UNSUPPORTED_GENERALIZATION:** span 2 comes from 'Models > OpenAI models > Hosted multi-agent (experimental) > Local function tools'. Its rejection rule holds on that surface, not in the ordinary Runner flow, where `needs_approval = True` correctly pauses for approval. The question as written is unqualified, so the composed answer is false in the default path.
- **SCOPE_IN_HEADING:** the qualification lives in the section heading, which §2D forbids relying on. It has to be inside the anchor or the case cannot carry it.
- **MULTI_HOP_STANDS:** with the scope inside span 2 the chain is real — see the semantic review in the internal review document.
- **CLAIM_SET:** the scope sentence 'All hosted agents share the model and tools configured for the request.' is inside the repaired span but is deliberately NOT an atomic claim. Quoting it would put 'the model' — whose antecedent is outside the span — into scored text, which turns a noncritical anaphora into a critical one. The span establishes the hosted-agent scope; the claims do not need to restate it.

REPAIRS MADE

- **E2 anchor extended** (evidence_scope_completion)

was 16756-16910, hash 2f49ae6be39cdc333394907d21b3cc5a25e24bd034782f78f478b26d95c87085
now 16313-16910, hash b0ef211a15b2f158ac09382986237d5050567eaf25439c000f7fc76a3415bafd

- **question rewritten**

was: If I set `needs_approval` to `True`, what happens?

now: If I set `needs_approval` to `True` on a function tool used with hosted agents, what happens?

- **answer rewritten**

was: Set `needs_approval` to `True` to always require approval or provide an async function that decides per call. SDK tool approval interruptions are not supported: any function tool whose `needs_approval` setting is not `False` is rejected before the request is sent.

now: The tool requires approval, but SDK tool-approval interruptions are not supported there, so a function tool whose `needs_approval` is not `False` is rejected before the request is sent.

- **composed_claim rewritten**

was: For `needs_approval`: Set `needs_approval` to `True` to always require approval or provide an async function that decides per call. Consequently, SDK tool approval interruptions are not supported: any function tool whose `needs_approval` setting is not `False` is rejected before the request is sent.

now: For hosted agents, a function tool with `needs_approval` set to `True` has a value that is not `False`, so it is rejected before the request is sent rather than pausing for approval.

- **composed_answer rewritten**

was: For `needs_approval`: Set `needs_approval` to `True` to always require approval or provide an async function that decides per call. Consequently, SDK tool approval interruptions are not supported: any function tool whose `needs_approval` setting is not `False` is rejected before the request is sent.

now: For hosted agents, a function tool with `needs_approval` set to `True` has a value that is not `False`, so it is rejected before the request is sent rather than pausing for approval.

- **bridge_relationship rewritten**

was: span 1 states a requirement or constraint on the bridge entity; span 2 states the behaviour that follows

now: Span 1 sets `needs_approval` to `True`; span 2 makes rejection conditional on `needs_approval` not being `False`. The chain runs through the value: `True` is not `False`, so the tool span 1 configures is the tool span 2 rejects.

- **why_span_1_alone_is_insufficient rewritten**

was: Span 1 establishes the condition on `needs_approval`, but does not state `needs_approval`.

now: Span 1 says a tool set to `True` always requires approval, and read alone it implies the run pauses for that approval — the opposite of what happens. It mentions neither hosted agents nor rejection, so a reader holding only span 1 answers the question wrongly rather than incompletely.

- **why_span_2_alone_is_insufficient rewritten**

was: Span 2 states what follows, but does not establish that it applies to `needs_approval`.

now: Span 2 says a function tool whose `needs_approval` is not `False` is rejected before the request is sent, but never says that `True` is a value anyone would set or what setting it is meant to achieve. A reader holding only span 2 knows an outcome without knowing the configuration that triggers it.

FLAGS FOR THE REVIEWER'S JUDGEMENT

- E2: noncritical anaphora — refers to 'the model' with no antecedent in the span. Nothing scored mentions 'model': the question, answer, claims and critical strings are all satisfied without resolving it.

8. Invariants

- No retrieval system was run against any candidate, in this phase or in generation. `retrieval_was_not_run = true`; `systems_executed = []`.
- SYSTEM-A `9afcb5b7...` and SYSTEM-B `304c3509...` remain frozen and unexecuted.

- The holdout is not frozen (`holdout_frozen = false`).
- Batches 001–003 are untouched: 53 `human_verified`, 53 `holdout_eligible`, 3 rejected.
- No batch-004 candidate is verified or eligible. All 15 owner decisions are `null` .

Generated by scripts/build_batch_004_review_pdf.py from gold_review_batch_004.json, gold_review_batch_004_repairs.json, gold_batch_004_qc.json, human_decisions_batch_004.json, GOLD-001-batch-004-internal-review.json, BATCH-004-near-miss-multihop-review.json, GOLD-001-batch-004-generation-report.json and GOLD-001-eligibility-status.json. Every count and every quoted span is read from those artifacts at build time. Batch 004, schema 1.2, corpus snapshot snap_689e336380a054d8039dc35b2c09cd0a, batch_sha256 e7c6d58c3c21b9ece940ae9665cba96a..., review 2026-08-21T16:41:19Z by claude (internal authoring review). Raw provider documentation is not redistributed; quoted spans are the short excerpts under review.